

# JSON Transformations

below I'll cover the **main Datagram transformations and JSON-specific transformations** you should know for Spark/Big Data interviews and projects.

Use:

```
from pyspark.sql import SparkSession
from pyspark.sql.functions import *
from pyspark.sql.types import *

spark = SparkSession.builder \
    .appName("JSON Transformations Practice") \
    .getOrCreate()

df = spark.read.json("users_large.json")
```

---

## Level 1 — Basic DataFrame Transformations

### Task 1 — Display the data

```
df.show()
```

### Task 2 — Display the schema

```
df.printSchema()
```

### Task 3 — Select specific columns

Select:

- user\_id
- name
- age
- city
- salary

```
df.select(
    "user_id",
    "name",
    "age",
    "city",
    "salary"
).show()
```

## Task 4 — Select nested JSON fields

Select:

- name
- area
- postal code

```
df.select(  
    "name",  
    "address.area",  
    "address.postal_code"  
) .show()
```

## Task 5 — Rename columns

Rename:

- name → full\_name
- salary → monthly\_salary

```
df.withColumnRenamed("name", "full_name") \  
  .withColumnRenamed("salary", "monthly_salary") \  
  .show()
```

---

# Level 2 — `withColumn()`

## Task 6 — Create a yearly salary column

```
df.withColumn(  
    "yearly_salary",  
    col("salary") * 12  
) .show()
```

## Task 7 — Create an age category

If age  $\geq$  30 → "Senior"

Otherwise → "Junior"

```
df.withColumn(  
    "age_category",  
    when(col("age") >= 30, "Senior")  
      .otherwise("Junior")  
) .show()
```

## Task 8 — Create salary category

```
df.withColumn(  
    "salary_category",  
    when(col("salary") >= 15000, "High")  
      .when(col("salary") >= 10000, "Medium")  
      .otherwise("Low")  
) .show()
```

## Task 9 — Add a constant column

```
df.withColumn(  
    "source",  
    lit("users_json")  
) .show()
```

---

# Level 3 — Filtering

## Task 10 — Find active users

```
df.filter(  
    col("is_active") == True  
) .show()
```

## Task 11 — Find users older than 30

```
df.filter(  
    col("age") > 30  
) .show()
```

## Task 12 — Find users from Cairo

```
df.filter(  
    col("city") == "Cairo"  
) .show()
```

## Task 13 — Find users from Cairo or Giza

```
df.filter(  
    col("city").isin("Cairo", "Giza")  
) .show()
```

## Task 14 — Multiple conditions

Find active users from Cairo earning more than 10,000.

```
df.filter(  
    (col("is_active") == True) &  
    (col("city") == "Cairo") &  
    (col("salary") > 10000)  
) .show()
```

---

## Level 4 — Sorting

### Task 15 — Sort by salary ascending

```
df.orderBy("salary").show()
```

### Task 16 — Sort by salary descending

```
df.orderBy(  
    col("salary").desc()  
) .show()
```

### Task 17 — Sort by department and salary

```
df.orderBy(  
    col("department"),  
    col("salary").desc()  
) .show()
```

---

## Level 5 — String Transformations

### Task 18 — Convert names to uppercase

```
df.withColumn(  
    "name_upper",  
    upper("name")  
) .show()
```

### Task 19 — Convert names to lowercase

```
df.withColumn(  
    "name_lower",  
    lower("name")  
) .show()
```

### Task 20 — Extract first name

```
df.withColumn(  
    "first_name",  
    split(col("name"), " ").getItem(0)  
).show()
```

### Task 21 — Extract last name

```
df.withColumn(  
    "last_name",  
    split(col("name"), " ").getItem(1)  
).show()
```

### Task 22 — Calculate name length

```
df.withColumn(  
    "name_length",  
    length("name")  
).show()
```

### Task 23 — Replace part of an email

```
df.withColumn(  
    "email",  
    regexp_replace("email", "@example.com", "@company.com")  
).show()
```

### Task 24 — Create username from email

```
df.withColumn(  
    "username",  
    split("email", "@").getItem(0)  
).show()
```

---

## Level 6 — Date Transformations

### Task 25 — Convert `join_date` to date

```
df.withColumn(  
    "join_date",  
    to_date("join_date")  
).show()
```

### Task 26 — Extract joining year

```
df.withColumn(  
    "join_year",  
    year("join_date")  
) .show()
```

### Task 27 — Extract joining month

```
df.withColumn(  
    "join_month",  
    month("join_date")  
) .show()
```

### Task 28 — Calculate years since joining

```
df.withColumn(  
    "years_since_joining",  
    year(current_date()) - year("join_date")  
) .show()
```

---

## Level 7 — Aggregations

### Task 29 — Count users

```
df.count()
```

### Task 30 — Average salary

```
df.select(  
    avg("salary").alias("average_salary")  
) .show()
```

### Task 31 — Maximum salary

```
df.select(  
    max("salary").alias("max_salary")  
) .show()
```

### Task 32 — Minimum salary

```
df.select(  
    min("salary").alias("min_salary")  
) .show()
```

### Task 33 — Total salaries

```
df.select(  
    sum("salary").alias("total_salary")  
) .show()
```

### Task 34 — Count users per city

```
df.groupBy("city") \  
    .count() \  
    .show()
```

### Task 35 — Average salary per department

```
df.groupBy("department") \  
    .agg(  
        avg("salary").alias("avg_salary")  
    ) \  
    .show()
```

### Task 36 — Multiple aggregations

```
df.groupBy("department").agg(  
    count("*").alias("employees"),  
    avg("salary").alias("avg_salary"),  
    min("salary").alias("min_salary"),  
    max("salary").alias("max_salary"),  
    sum("salary").alias("total_salary")  
) .show()
```

---

## Level 8 — `groupBy()` + `filter()`

### Task 37

Find departments whose average salary is greater than 10,000.

```
df.groupBy("department") \  
    .agg(  
        avg("salary").alias("avg_salary")  
    ) \  
    .filter(  
        col("avg_salary") > 10000  
    ) \  
    .show()
```

---

## Level 9 — Null Handling

Even though this dataset currently has very few/no nulls, you should practice the operations.

### Task 38 — Count nulls

```
df.select([
    count(
        when(col(c).isNull(), c)
    ).alias(c)
    for c in df.columns
]).show()
```

### Task 39 — Drop rows containing nulls

```
df.dropna().show()
```

### Task 40 — Fill null values

```
df.fillna({
    "salary": 0,
    "city": "Unknown"
}).show()
```

---

## Level 10 — Duplicate Handling

### Task 41 — Remove duplicate users

```
df.dropDuplicates().show()
```

### Task 42 — Remove duplicates based on `user_id`

```
df.dropDuplicates(["user_id"]).show()
```

---

## Level 11 — Array Transformations

This is where the JSON dataset becomes really useful.

Remember:

```
skills = ["Python", "SQL", "Spark"]
```



### Task 43 — Display skills

```
df.select(
    "name",
    "skills"
).show(truncate=False)
```

### Task 44 — Get first skill

```
df.select(
    "name",
    col("skills")[0].alias("first_skill")
).show()
```

### Task 45 — Get number of skills

```
df.select(
    "name",
    size("skills").alias("number_of_skills")
).show()
```

### Task 46 — Check whether user knows Python

```
df.select(
    "name",
    array_contains("skills", "Python").alias("knows_python")
).show()
```

### Task 47 — Explode skills

```
df.select(
    "name",
    explode("skills").alias("skill")
).show()
```

Result conceptually:

|                |        |
|----------------|--------|
| Ahmed Ali      | Python |
| Ahmed Ali      | SQL    |
| Ahmed Ali      | Spark  |
| Youssef Hassan | Python |
| Youssef Hassan | Kafka  |
| ...            |        |

### Task 48 — `explode_outer()`

```
df.select(
    "name",
    explode_outer("skills").alias("skill")
).show()
```

---



## Level 12 — Nested JSON Objects

Your dataset contains:

```
address
├── street
├── area
└── postal_code
```

### Task 49 — Select the entire address

```
df.select(
    "name",
    "address"
).show(truncate=False)
```

### Task 50 — Select individual nested fields

```
df.select(
    "name",
    col("address.street"),
    col("address.area"),
    col("address.postal_code")
).show()
```

### Task 51 — Create a full address

```
df.withColumn(
    "full_address",
    concat_ws(
        ", ",
        "address.street",
        "address.area",
        "address.postal_code"
    )
).show(truncate=False)
```

---



## Level 13 — Nested Arrays

Your dataset contains:

```
orders
[
  {
    order_id,
    product,
    quantity,
    price
  }
]
```

```
    },  
    ...  
]
```

## Task 52 — Display orders

```
df.select(  
    "name",  
    "orders"  
) .show(truncate=False)
```

## Task 53 — Get first order

```
df.select(  
    "name",  
    col("orders")[0].alias("first_order")  
) .show(truncate=False)
```

## Task 54 — Get first order's product

```
df.select(  
    "name",  
    col("orders")[0]["product"].alias("product")  
) .show()
```

## Task 55 — Count orders

```
df.select(  
    "name",  
    size("orders").alias("order_count")  
) .show()
```

---



# Level 14 — Explode Orders

## Task 56 — Explode orders

```
orders_df = df.select(  
    "user_id",  
    "name",  
    explode("orders").alias("order")  
)  
  
orders_df.show(truncate=False)
```

Now:

```
order
├── order_id
├── product
├── quantity
└── price
```

## Task 57 — Extract order fields

```
orders_df.select(
    "user_id",
    "name",
    col("order.order_id").alias("order_id"),
    col("order.product").alias("product"),
    col("order.quantity").alias("quantity"),
    col("order.price").alias("price")
).show()
```

---



# Level 15 — Calculate Order Values

## Task 58 — Calculate total order value

```
orders_df = orders_df.withColumn(
    "total",
    col("order.quantity") * col("order.price")
)

orders_df.show()
```

## Task 59 — Find expensive orders

```
orders_df.filter(
    col("total") > 10000
).show()
```

## Task 60 — Calculate total spending per user

```
orders_df.groupBy(
    "user_id",
    "name"
).agg(
    sum(
        col("order.quantity") * col("order.price")
    ).alias("total_spending")
).show()
```

---



## Level 16 — Array Functions

### Task 61 — Sort skills

```
df.select(  
    "name",  
    sort_array("skills").alias("sorted_skills")  
) .show(truncate=False)
```

### Task 62 — Reverse skills

```
df.select(  
    "name",  
    reverse("skills").alias("reversed_skills")  
) .show(truncate=False)
```

### Task 63 — Join skills into a string

```
df.select(  
    "name",  
    concat_ws(", ", "skills").alias("skills_string")  
) .show()
```

---



## Level 17 — JSON Functions

Now we get specifically into **JSON transformations**.

### Task 64 — Convert a struct to JSON

Take the `address` struct and convert it to JSON.

```
df.select(  
    "name",  
    to_json("address").alias("address_json")  
) .show(truncate=False)
```

You'll get something like:

```
{"street": "Nile Street", "area": "Maadi", "postal_code": "11728"}
```

---

### Task 65 — Convert orders to JSON

```
df.select(  
    "name",  
    to_json("orders").alias("orders_json")
```

```
).show(truncate=False)
```

---

## Level 18 — `from_json()`

This is **VERY important for Kafka + Spark Structured Streaming**.

Suppose you have:

```
json_string
```

Create a DataFrame containing JSON strings:

```
json_df = spark.createDataFrame([
    ('{"id":101,"name":"Ahmed","age":25}',),
    ('{"id":102,"name":"Youssef","age":28}',),
], ["json_string"])
```

Define schema:

```
schema = StructType([
    StructField("id", IntegerType()),
    StructField("name", StringType()),
    StructField("age", IntegerType())
])
```

### Task 66 — Parse JSON

```
parsed_df = json_df.withColumn(
    "data",
    from_json("json_string", schema)
)
```

```
parsed_df.show(truncate=False)
```

### Task 67 — Extract parsed fields

```
parsed_df.select(
    "data.id",
    "data.name",
    "data.age"
).show()
```

This is the pattern you'll use with Kafka:

```
Kafka value
  ↓
cast("string")
  ↓
from_json()
```

```
↓  
StructType  
↓  
select("data.*")
```

---

## Level 19 — `selectExpr()`

### Task 68

Use SQL expressions instead of `select()`.

```
df.selectExpr(  
    "name",  
    "age",  
    "salary",  
    "salary * 12 AS yearly_salary"  
).show()
```

### Task 69 — Use SQL expressions for filtering

```
df.selectExpr(  
    "name",  
    "department",  
    "salary"  
).filter(  
    "salary > 10000"  
).show()
```

---

## Level 20 — `alias()`

### Task 70

Rename expressions using `alias()`.

```
df.select(  
    col("name").alias("employee_name"),  
    col("salary").alias("monthly_salary"),  
    (col("salary") * 12).alias("yearly_salary")  
).show()
```

---

## Level 21 — Partition Transformations

These are important for Spark performance.

## Task 71 — Check number of partitions

```
df.rdd.getNumPartitions()
```

## Task 72 — Repartition

```
df2 = df.repartition(4)
```

## Task 73 — Repartition by city

```
df2 = df.repartition(  
    4,  
    "city"  
)
```

## Task 74 — Coalesce

```
df3 = df.coalesce(2)
```

Remember:

```
repartition()  
  ↓  
can increase/decrease partitions  
usually causes shuffle
```

```
coalesce()  
  ↓  
usually decreases partitions  
avoids full shuffle
```

---

# Level 22 — Set Operations

Create two DataFrames:

```
df1 = df.filter(col("age") > 30)  
df2 = df.filter(col("salary") > 15000)
```

## Task 75 — Union

```
df1.union(df2).show()
```

## Task 76 — Union by column name

```
df1.unionByName(df2).show()
```



## Task 77 — Distinct

```
df1.union(df2).distinct().show()
```

---

# Level 23 — Window Transformations

Very important for interviews.

## Task 78 — Rank employees by salary

```
from pyspark.sql.window import Window

window_spec = Window.orderBy(
    col("salary").desc()
)

df.withColumn(
    "rank",
    rank().over(window_spec)
).show()
```

## Task 79 — Rank employees inside each department

```
window_spec = Window.partitionBy(
    "department"
).orderBy(
    col("salary").desc()
)

df.withColumn(
    "rank",
    rank().over(window_spec)
).show()
```

## Task 80 — row\_number()

```
df.withColumn(
    "row_number",
    row_number().over(window_spec)
).show()
```

## Task 81 — dense\_rank()

```
df.withColumn(
    "dense_rank",
    dense_rank().over(window_spec)
).show()
```

---



## Level 24 — Conditional Transformations

### Task 82 — Categorize users

```
df.withColumn(  
    "level",  
    when(col("salary") >= 15000, "Senior")  
      .when(col("salary") >= 10000, "Mid-Level")  
      .otherwise("Junior")  
) .show()
```

### Task 83 — Multiple conditions

```
df.withColumn(  
    "employee_type",  
    when(  
        (col("age") >= 30) &  
        (col("salary") >= 12000),  
        "Experienced"  
    )  
    .when(  
        col("age") < 25,  
        "Young"  
    )  
    .otherwise("Regular")  
) .show()
```

---



## Level 25 — Final Mini Project

Now combine everything.

### Task 84

Create a final employee dataset containing:

- user\_id
- first\_name
- last\_name
- city
- department
- age
- salary
- yearly\_salary
- salary\_category
- join\_year
- number\_of\_skills

- number\_of\_orders
- full\_address

## Solution

```
final_df = (
    df
    .withColumn(
        "first_name",
        split("name", " ").getItem(0)
    )
    .withColumn(
        "last_name",
        split("name", " ").getItem(1)
    )
    .withColumn(
        "yearly_salary",
        col("salary") * 12
    )
    .withColumn(
        "salary_category",
        when(col("salary") >= 15000, "High")
        .when(col("salary") >= 10000, "Medium")
        .otherwise("Low")
    )
    .withColumn(
        "join_year",
        year("join_date")
    )
    .withColumn(
        "number_of_skills",
        size("skills")
    )
    .withColumn(
        "number_of_orders",
        size("orders")
    )
    .withColumn(
        "full_address",
        concat_ws(
            ", ",

```



## What this dataset lets you practice

You now have practice for:

| Category  | Functions / Transformations                                                         |
|-----------|-------------------------------------------------------------------------------------|
| Selection | <code>select()</code> , <code>selectExpr()</code>                                   |
| Columns   | <code>withColumn()</code> , <code>withColumnRenamed()</code> , <code>alias()</code> |
| Filtering | <code>filter()</code> , <code>where()</code>                                        |

| Category          | Functions / Transformations                                                                                                             |
|-------------------|-----------------------------------------------------------------------------------------------------------------------------------------|
| Sorting           | <code>orderBy()</code> , <code>sort()</code>                                                                                            |
| Conditions        | <code>when()</code> , <code>otherwise()</code>                                                                                          |
| Constants         | <code>lit()</code>                                                                                                                      |
| Strings           | <code>upper()</code> , <code>lower()</code> , <code>length()</code> , <code>split()</code> , <code>regexp_replace()</code>              |
| Dates             | <code>to_date()</code> , <code>year()</code> , <code>month()</code> , <code>current_date()</code>                                       |
| Aggregation       | <code>count()</code> , <code>sum()</code> , <code>avg()</code> , <code>min()</code> , <code>max()</code>                                |
| Grouping          | <code>groupBy()</code> , <code>agg()</code>                                                                                             |
| Nulls             | <code>dropna()</code> , <code>fillna()</code> , <code>isNull()</code>                                                                   |
| Duplicates        | <code>dropDuplicates()</code> , <code>distinct()</code>                                                                                 |
| Arrays            | <code>size()</code> , <code>explode()</code> , <code>explode_outer()</code> , <code>array_contains()</code> , <code>sort_array()</code> |
| Structs           | Nested field access                                                                                                                     |
| JSON              | <code>from_json()</code> , <code>to_json()</code>                                                                                       |
| Nested data       | Struct + Array + Array of Struct                                                                                                        |
| Partitions        | <code>repartition()</code> , <code>coalesce()</code>                                                                                    |
| Set operations    | <code>union()</code> , <code>unionByName()</code>                                                                                       |
| Windows           | <code>row_number()</code> , <code>rank()</code> , <code>dense_rank()</code>                                                             |
| Conditional logic | <code>when()</code> / <code>otherwise()</code>                                                                                          |

## ★ Most important for your Spark Streaming/Kafka goal

Pay extra attention to these:

```
from_json()
to_json()
StructType
StructField
ArrayType
explode()
explode_outer()
nested structs
nested arrays
select()
withColumn()
filter()
groupBy()
agg()
```

Especially this pipeline:

```
Kafka
↓
value
↓
cast("string")
```

↓  
from\_json()  
↓  
StructType  
↓  
nested JSON  
↓  
explode()  
↓  
transformations  
↓  
aggregation  
↓  
output

That is the bridge between the **JSON practice you're doing now** and the **Kafka + Spark Structured Streaming projects** you want to build.